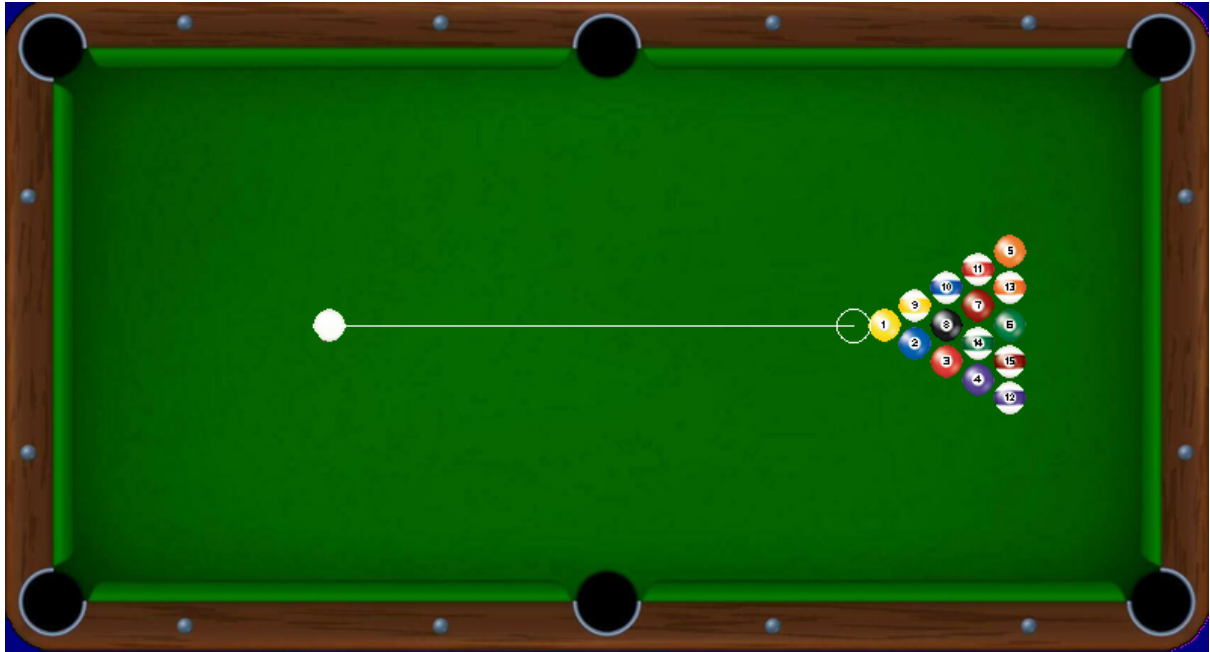


Real-time systems project



Emilio Gigante

Matricola: 559350

Email: e.gigante1@studenti.unipi.it

LM Ingegneria Robotica e dell'Automazione

Informatica e sistemi in tempo reale

Academic Year 2024/2025

1 Introduction

The purpose of this work is to develop a program written in C language that creates a real-time simulation of the game of 8 ball pool.

This was achieved through some tasks regulated by a library created using the pthread library.

2 Game rules

In order to make all the following clear, here is a summary of the 8 ball pool rules:

There are 2 opponents that can interact only with a white marble, through which they must strike other marbles (the fact that the player has to hit the white marble to hit the other will be taken for granted from now on).

The first instance of the game is called **break** in which we have the white marble on one side of the table and all the other marbles on the other side arranged in a triangle formation. To perform the break correctly, a player has to hit the closer corner of the triangle hard enough to make at least 4 marbles touch the game field banks, otherwise the break has to be repeated by the other player. If during the break a marble is pocketed the player who performed the break keeps playing, otherwise the turn passes to the other player.

After the break there is an **open game phase** in which both players have to pocket any marble, but the white one and the 8 one. There are 2 groups of marbles: **solid** (from 1 to 7) and **striped** (from 9 to 15). The first player who pockets a ball claims the relative group and will be allowed to hit only its group of marbles with its strike.

When the groups are assigned we have a **standard game phase** in which the player has to pocket every marble of its group. When all of the marbles of the group are pocketed the player has to pocket the **8 marble**, which is neither solid or striped. The pocketing of the 8 marble must be preceded by the declaration of the hole in which the player wants to pocket the marble. If the player succeeds in doing that, wins the game.

A **foul** is an irregularity in the game that results in a change of turn and the placement of the white marble by the opponent in the table position he prefers. actions that cause a foul are:

- Not hitting any marble.
- Hitting the 8 marble first, before pocketing the other requested marbles.
- Hitting a marble belonging to the opponent's group first.
- Pocketing the white marble.

Some actions can cause the victory of the opponent:

- Pocketing the 8 marble before the other requested marbles.
- Pocketing the 8 marble in a different hole as declared before the strike.
- Pocketing the white marble in the same turn as the 8 marble.

3 Shared data structure

The most important element of the program is the array of structures that defines the status of every ball (as marble), each structure has these fields:

- position: x and y [m]
- velocity: v_x and v_y [m/s]
- activity flag
- type (as group)
- phase in which it was eliminated (if eliminated)
- ball position when it's eliminated [fixed]
- relative bitmap for graphic [fixed]
- trail color [fixed]

Other structures used are one that defines the position of the holes in the game field and one that saves the previous positions of each ball to draw a trail.

A pair of important elements for the shot (as strike) event are:

- The angle of the shot with respect to the position of the white ball
- The module of the velocity of the white ball immediately after the shot

There are 3 parameters the user can change manually, their meaning will be explained further in this report: **friction** factor, **dumping** factor and **time scale** factor.

Turns are defined by an element that serves as a condition that all the balls are still (enough) which enables players to play in their turn, this will also be explained better further on.

Counters are also useful, and also useful are store variables that allow the program to confront the differences of these counters from one turn to another.

There are 3 counters for hitting a solid ball, a striped ball and the 8 ball and other 3 for pocketing them.

There are also some flags that are important to make order in the game development, every flag defines the following game situations:

- trail save and displaying
- player turn
- phase of the game (break, open game, standard game)
- type assignment
- making sure that the counter of collisions counts just the first of every turn
- phase switch
- turn switch
- foul occurrence
- possible winning turn for both players
- victory assignment
- declared hole index during a winning turn

4 Tasks

Here is a description of the 5 tasks that rule the game and how they were built.

4.1 ball task

This is the task that rules the updating of each ball status, their interaction with the table and each other. The task period was defined as a global constant of the value of 40 ms (same for the deadline) which was chosen empirically as a compromise between realistic ball behaviour and the need to allow the task to perform all functions in the period time and thus avoid deadline misses which in most of the cases cause irregularity in the collisions. The priority of the task is set to 90 which is the higher value among all tasks.

The whole task works under a semaphore because it modifies the ball structure which has to be used also by the displaying function that will be shown further on. It is important to analyze the functions that this task uses.

4.1.1 update status

Position is tied to velocity by a simple Euler integration, on the other hand velocity is modified only by friction also using the task loop.

$$x_{k+1} = x_k + v_{x,k}\Delta t$$

$$y_{k+1} = y_k + v_{y,k}\Delta t$$

$$v_{x,k+1} = (1 - f)v_{x,k}$$

$$v_{y,k+1} = (1 - f)v_{y,k}$$

Where f is the friction factor mentioned before, so the higher the factor the rougher the table.

While Δt is the product of the task period and the time scale factor mentioned previously, this way decreasing the factor allows us to have a smaller integration step but also a more intense friction phenomenon, the opposite happens if we increase the factor.

4.1.2 handle bounce

This function keeps the balls inside the table and takes into account the loss of energy of the balls caused by impact with the table bank.

Take, for example, the case of a ball hitting the left-hand edge of the table: if the ball went beyond the x-value defining the edge, it would be driven back onto that axis of the x-value that exceeded the limit and the x-component would change sign and be multiplied by a dumping factor η discussed above, the higher the factor the lower the energy loss.

$$v' = -\eta v$$

This happens in a similar way for the right margin and likewise with y for upper and bottom margin.

While the central holes on the long sides are directly accessible, to access the holes in the corners, a short tunnel must be provided to allow the ball to reach the hole. The walls of these tunnels are set at a 45° angle and function similarly to the horizontal and vertical walls, but without position correction.

The function also increases a bounce counter that is useful for defining a correct break.

4.1.3 handle collisions

Being the game all about collisions this function is the key of the game. It takes in analysis a couple of balls at the time and computes their distance to check if any collision happened.

If a collision is detected the first thing that is done is to increase the collision counters if the white ball hit a solid or striped ball or the 8 ball.

When the collision occurs the function computes the x and y components of the normal vector of the direction of connecting the centers of the balls and a perpendicular tangential vector. After that the virtual interpenetration of the bodies is computed and the balls are distanced by half of this value because we want them to behave like rigid bodies.

For the velocities we use the solution of a partially inelastic collision problem of bodies with same mass in 1D, because only the normal component of the velocity is affected while the tangential one remains unchanged:

$$\begin{cases} v_i + v_j = v'_i + v'_j \\ \eta^2(v_i^2 + v_j^2) = v'^2_i + v'^2_j \end{cases}$$

In this case η is the dumping factor discussed for the bounce function.

Only one of the solutions of this system is physically meaningful, but still an important differentiation has to be done, depending on the value of this element:

$$\Delta = (2\eta^2 - 1)(v_i^2 + v_j^2) - 2v_iv_j$$

if $\Delta > 0$ we have, as expected, a partially inelastic impact following this law:

$$v'_i = 0.5(v_i + v_j) + \sqrt{\Delta}$$

$$v'_j = 0.5(v_i + v_j) - \sqrt{\Delta}$$

Otherwise the impact is considered completely inelastic and the velocity of both bodies is the average of the velocities before impact.

4.1.4 handle holes

This function allows the game to deal coherently with the consequences of any ball ending up in a hole. It has the possibility of increasing the hole counters and assigns to the players a foul and even the victory.

4.1.5 wake store

This function stores the previous positions of the ball in a wake in order to allow the drawing function that will be shown further to display the trail of every ball.

4.2 shot task

This is the task that rules the strike event. In particular when the game is stationary, meaning no ball moves, the player can direct the shot by pressing and holding the mouse wheel and moving the mouse around the field, the task will compute the angle of the line passing through the white ball center and the mouse position with respect to horizontal.

The player can also regulate the velocity the white ball must have as a consequence of the strike, the left button will increase the velocity and the right one will decrease it.

When both direction and velocity are set the player has to perform the strike by pressing the space bar. The period of the task was chosen (as well as the deadline) as 20 ms which is a value that allows a fast reaction to the player command, especially about the strike moment, the simplicity of the task and the fact that it works only when the game is stationary allows to have a correct behaviour despite possible deadline misses. The priority of this task is set to 70, which is the third higher value among all tasks.

4.3 set param task

This task allows to regulate 3 parameters by the keyboard, but just when the game is stationary.

In particular by pressing Q we can increase the friction factor, and decrease it with A; increase dumping factor by pressing W and decrease it with S; increase the time scale factor by pressing E and decrease it with D.

Since the task is active only when the game is stationary, and the current values of the parameters are visible on the user interface, deadline misses are basically not harmful.

The period and the deadline were chosen as 20 ms to provide a fast response to the user command.

The priority of the task is set to 60, which puts it to the 4° place among the priorities.

4.4 manage task

This task makes sure that the game is played according to its rules, to do that it mostly uses the counters and the flags that were described before 3. Priority is set to 80, which is the second higher value among all the tasks. The period and deadline of this task are set to a consistently larger value with respect to the other tasks: 200 *ms*. That is because this task only works at the beginning of the player turn and it is important that all the duties are accomplished to avoid unexpected or incorrect behaviour, so it is important to avoid deadline misses, also on the other hand there is little need for a fast response on the few functions that need the user interaction, function that will now be discussed.

4.4.1 foul

When a foul occurs the opponent can position the ball wherever it wants, so the white ball will be deactivated and by moving the mouse around the field it will be possible to see the white ball moving around accordingly. To make evident that we are in a foul occurrence the white ball contour will turn red. When the user is satisfied with the ball position it just has to press TAB to make the white ball playable again.

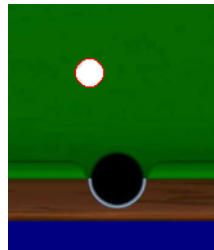


Figure 1: The white ball after a foul was committed

4.4.2 hole declaration

When either of the player has to take a shot for the win needs to declare the hole in which he wants to pocket the 8 ball, so at the beginning of the turn the player will see the currently declared hole, which will be highlighted with a red dot. the player can change that using the left and right arrows of the keyboard, and confirm its choice by pressing ENTER, at that point the player can perform the winning shot.



Figure 2: The highlighted declared hole

4.4.3 endgame

This function checks the win flag and assigns the victory to the right player by covering the game with the coherent victory bitmap, which will also give the user the possibility to restart the game by pressing ENTER, by doing so the balls positions and all the necessary flags will be reset to the initial value.

4.5 display task

This task takes information from the shared data to create a graphic output of the game development on screen using the Allegro library.

The table and all the balls are bitmaps, the balls are pasted on the game table in the correct position and then the table is pasted on screen, also the empty table bitmap is used to erase the previous step of the game.

Using the previously mentioned wake 3, pixels are used to draw balls trails.

This task also shows the white ball trajectory and the shot power indicator, and outside the table are also shown the indicators of the parameters editable by the set param task 4.3, the player and the trail indicator.

The semaphore used in the ball task 4.1 was used to protect a brief section where position data from the ball structure is copied in a local variable.

This task priority is set to 50, the least among all tasks.

Period and deadline are set to 50 *ms* which is a little larger than the ball task period, that is because i needed to reduce the deadline misses and this is the largest value that allows an acceptable visual representation of the game.

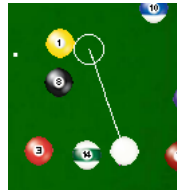


Figure 3: Game graphic representation

5 User interface

Parameters that define the dimensions in pixels of the table and balls bitmaps are written as global constants, then constant variables are used to calculate the coordinates for having the table at the center of the window.

Over the table we have an info box where all the commands that the user has are written.

On the left side of the table we have the shot power indicator that is visible just when the game is stationary.

Below that there are 2 boxes, one shows whose the turn to play and the other shows if the wake for trail depiction is being saved and shown.

Below the table there are the indicators of the set param task and the instructions to modify them.

Besides this we have a deadline misses counter for all the tasks.

On the right side of the table the balls that were eliminated will be visible just to help the user during the game.



Figure 4: Graphical interface of the game

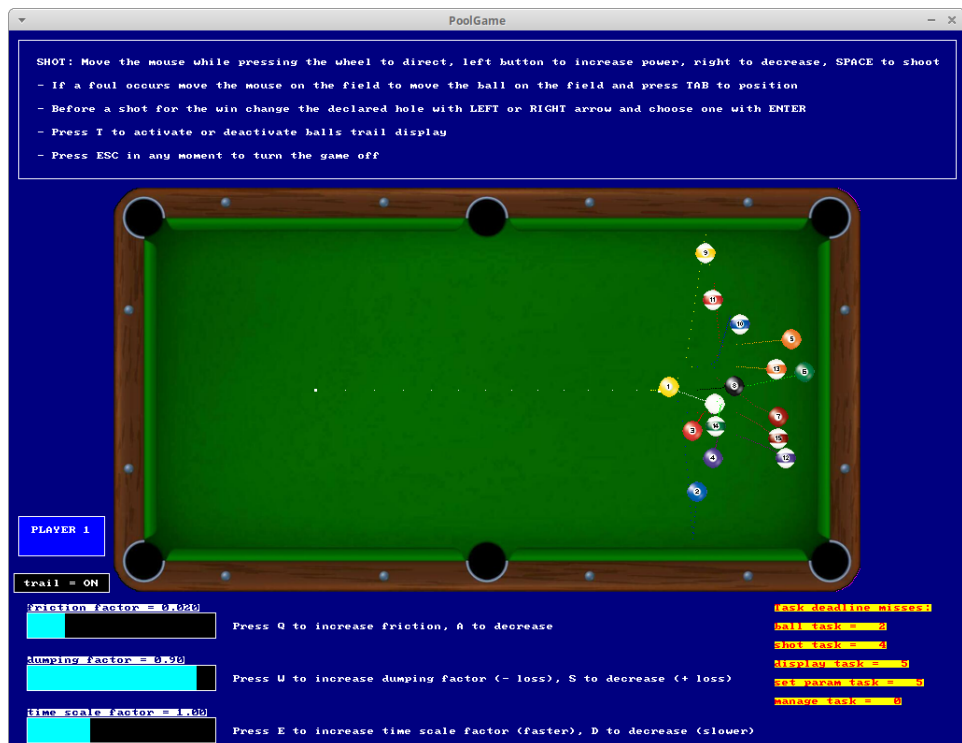


Figure 5: Break moment with the trail option active

6 Task-resource diagram

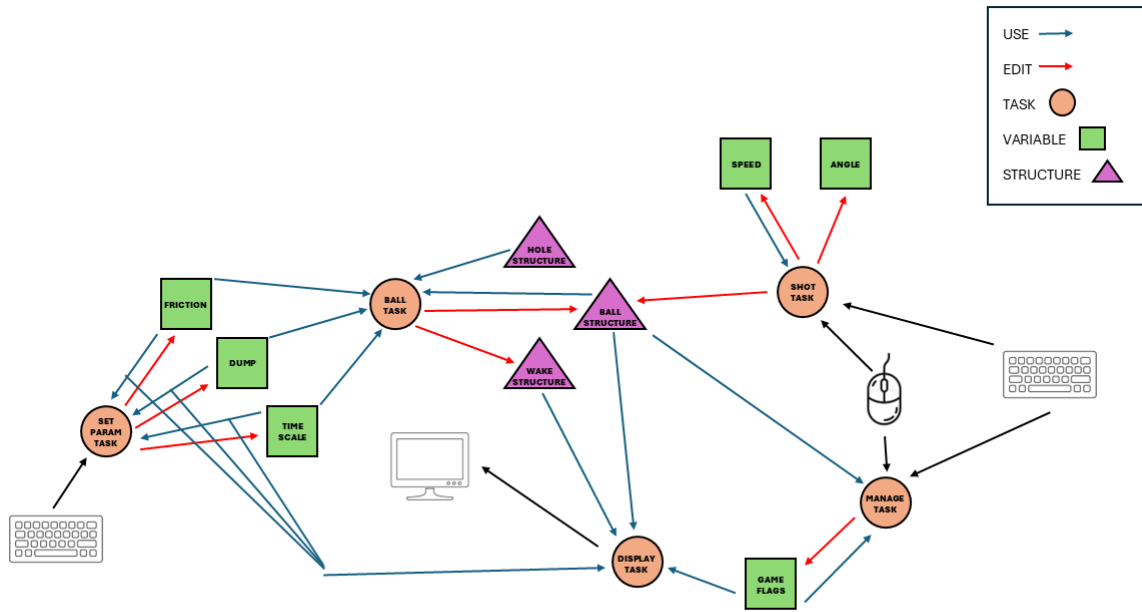


Figure 6: This diagram shows which tasks use which resources and how the user interacts with the tasks.